



PKCERT AI & Software Development Internship

Name	Abdul Rehman
Course	AI & Software Development
Due Date	7/7/2026

Part A

Q1 Demonstrate Python Data Types and Explain Their Uses

Definition: Python data types define the type of value a variable can store.

Python Code

```
age=21
gpa=3.85
is_passed=True
name='Ali Raza'
subjects=['Python','AI','ML']
coordinates=(33.6844,73.0479)
student={'name':name,'age':age,'gpa':gpa}
unique_skills={'Python','SQL','Git'}
print(age,gpa,is_passed,name)
print(subjects)
print(coordinates)
print(student)
print(unique_skills)
```

Output

```
Ali Raza
21
3.85
True
['Python', 'AI', 'ML']
(33.6844, 73.0479)
{'name': 'Ali Raza', 'age': 21, 'gpa': 3.85}
{'Git', 'SQL', 'Python'}
PS C:\Users\abdul\OneDrive\Desktop\TASK 02 INTERNSHIP> █
```

Data Type	Use
Integer (<code>int</code>)	Stores whole numbers.
Float (<code>float</code>)	Stores decimal numbers.
String (<code>str</code>)	Stores text.

Boolean (bool)	Stores True or False values.
List (list)	Stores multiple items in order.
Tuple (tuple)	Stores fixed (unchangeable) items.
Dictionary (dict)	Stores data as key-value pairs.
Set (set)	Stores unique values.

Q2 User-defined Functions

Definition: A user-defined function accepts parameters and returns a value.

Python Code

```
def calculate_average(marks):
    return sum(marks)/len(marks)
marks=[88,92,79,95]
print(calculate_average(marks))
```

Output

```
PS C:\Users\abdu1\OneDrive\Desktop\TASK 02 INTERNSHIP> & D:\Python\python.exe ERNSHIP/Part_A.py"
Average marks : 88.5
PS C:\Users\abdu1\OneDrive\Desktop\TASK 02 INTERNSHIP> █
```

Function to calculate the area of a rectangle

```
def calculate_area(length, width):

    area = length * width

    return area

length = 8

width = 5

print("Area =", calculate_area(length, width))
```

Output:

```
PS C:\Users\abdu1\OneDrive\Desktop\TASK 02 INTERNSHIP> & D:\Python\python.exe "c:/Users/abdu1/OneDrive/Desktop/TASK 02 INT
ERNSHIP/Part_A.py"
Area = 40
█
```

Q3 Comprehensions

Definition: List and dictionary comprehensions provide a concise way to create lists and dictionaries in a single line of code.

Python Code

```
numbers=range(1,11)
even_squares=[n**2 for n in numbers if n%2==0]
print(even_squares)
squares_dict={n:n**2 for n in range(1,6)}
print(squares_dict)
```

```
PS C:\Users\abdul\OneDrive\Desktop\TASK 02 INTERNSHIP> &
ERNSHIP/Part_A.py"
-----LIST COMPREHENSION-----
[4, 16, 36, 64, 100]
*****
-----DICTIONARY COMPREHENSION-----
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
PS C:\Users\abdul\OneDrive\Desktop\TASK 02 INTERNSHIP>
```

Output

Benefits

- Less code
- Easy to read
- Faster execution
- Better code readability

Q4 OOP

Definition: OOP uses classes and objects.

OOP Concepts Used

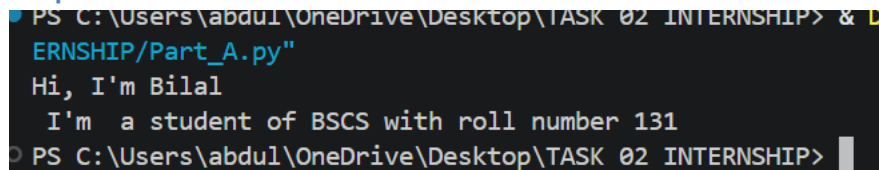
- **Class:** Blueprint for creating objects.
- **Object:** Instance of a class.

- **Constructor:** Initializes object data.
- **Inheritance:** Allows one class to inherit another class's properties and methods.

Python Code

```
class Person:
    def __init__(self,name,age):
        self.name=name
        self.age=age
    def introduce(self):
        return f'Hi, I'm {self.name}'
class Student(Person):
    def __init__(self,name,age,roll,program):
        super().__init__(name,age)
        self.program=program
p=Person('Bilal',40)
print(p.introduce())
```

Output



```
PS C:\Users\abdul\OneDrive\Desktop\TASK 02 INTERNSHIP> & U
ERNSHIP/Part_A.py
Hi, I'm Bilal
I'm a student of BSCS with roll number 131
PS C:\Users\abdul\OneDrive\Desktop\TASK 02 INTERNSHIP>
```

Part B

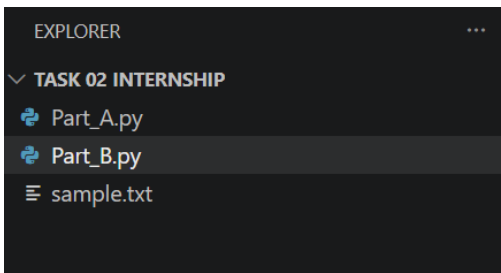
Q1 Text File Handling

Definition: Read, write and append data to a text file.

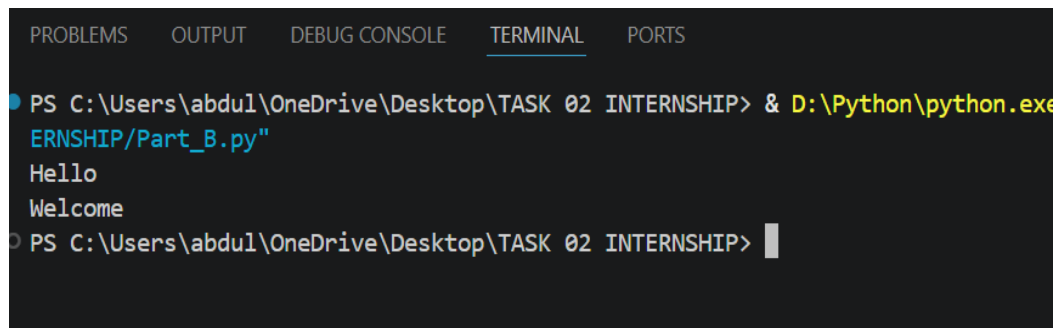
Python Code

```
with open('sample.txt','w') as f:
    f.write('Hello')
with open('sample.txt','a') as f:
    f.write('\nWelcome')
with open('sample.txt') as f:
    print(f.read())
```

File Write:



File Read:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\abdul\OneDrive\Desktop\TASK 02 INTERNSHIP> & D:\Python\python.exe ERNSHIP/Part_B.py"
Hello
Welcome
PS C:\Users\abdul\OneDrive\Desktop\TASK 02 INTERNSHIP>
```

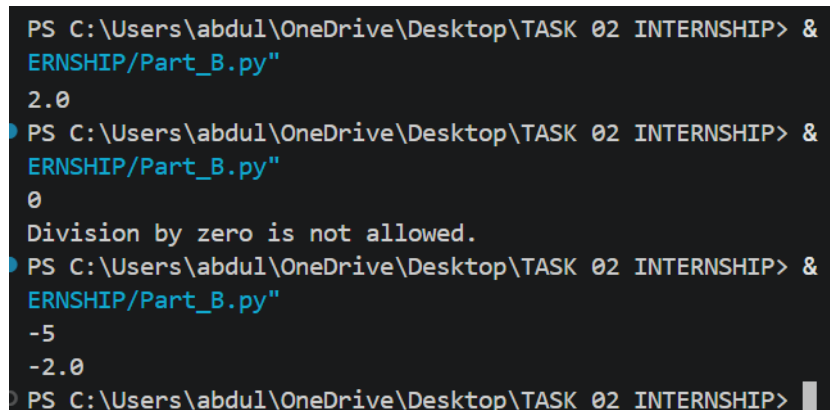
Q2 Exception Handling

Definition: Handles errors without stopping the program.

Python Code

```
try:
    print(10/int(input()))
except ZeroDivisionError:
    print('Division by zero is not allowed.')
except ValueError:
    print('Enter valid number.')
```

Output



```
PS C:\Users\abdul\OneDrive\Desktop\TASK 02 INTERNSHIP> & ERNSHIP/Part_B.py"
2.0
PS C:\Users\abdul\OneDrive\Desktop\TASK 02 INTERNSHIP> & ERNSHIP/Part_B.py"
0
Division by zero is not allowed.
PS C:\Users\abdul\OneDrive\Desktop\TASK 02 INTERNSHIP> & ERNSHIP/Part_B.py"
-5
-2.0
PS C:\Users\abdul\OneDrive\Desktop\TASK 02 INTERNSHIP>
```

Q3 CSV and JSON

Definition: CSV (Comma-Separated Values) and JSON (JavaScript Object Notation) are common file formats used to store and exchange data. Python provides built-in modules to read and display data from these files.

Python Code

CODE TO READ AND DISPLAY DATA FROM CSV#

```
import csv

print("Student Data from CSV File:\n")

with open("students.csv", "r") as file:

    reader = csv.reader(file)

    for row in reader:

        print(row)
```

```
● PS C:\Users\abdul\OneDrive\Desktop\TASK 02 INTERNSHIP> &
  ERNSHIP/Part_B.py"
  Student Data from CSV File:

  ['Name', 'Age', 'Course']
  ['Ali', '22', 'Python']
  ['Ayesha', '21', 'AI']
  ['Bilal', '23', 'Machine Learning']
○ PS C:\Users\abdul\OneDrive\Desktop\TASK 02 INTERNSHIP>
```

CODE TO READ AND DISPLAY DATA FROM JSON FILE

```
import json

print("Student Data from JSON File:\n")

with open("student.json", "r") as file:

    data = json.load(file)

print(data)
```

```
● PS C:\Users\abdul\OneDrive\Desktop\TASK 02 INTERNSHIP>
  ERNSHIP/Part_B.py"
  Student Data from JSON File:

  {'Name': 'Ahmed', 'Age': 20, 'Grade': 'A'}
○ PS C:\Users\abdul\OneDrive\Desktop\TASK 02 INTERNSHIP>
```

Great! Since **Parts C, D, and E are theory-based**, you don't need any code. You can copy and paste the following directly into your Word document.

Part C – Development Environment (10 Marks)

Question: Prepare a one-page report on Virtual Environments, Conda Environments, and Dependency Isolation.

Definition

A development environment is a setup that helps developers write, test, and manage software efficiently. Virtual environments and Conda environments allow each project to have its own packages and dependencies.

Virtual Environment

A virtual environment is an isolated Python environment created using the `venv` module. It allows each project to use its own Python packages without affecting other projects.

Advantages

- Keeps project dependencies separate.
 - Prevents package version conflicts.
 - Makes projects easier to manage.
 - Helps create reproducible development environments.
-

Conda Environment

A Conda environment is created using the Conda package manager. It can manage Python packages as well as non-Python libraries and allows different Python versions for different projects.

Advantages

- Manages Python and non-Python packages.
 - Supports multiple Python versions.
 - Easy package installation and updates.
 - Commonly used in Data Science and Machine Learning.
-

Dependency Isolation

Dependency isolation means keeping each project's libraries and package versions separate from other projects. This prevents conflicts and ensures that each project works correctly with its required dependencies.

Benefits

- Avoids package conflicts.
- Improves project stability.
- Makes collaboration easier.
- Simplifies deployment to other systems.

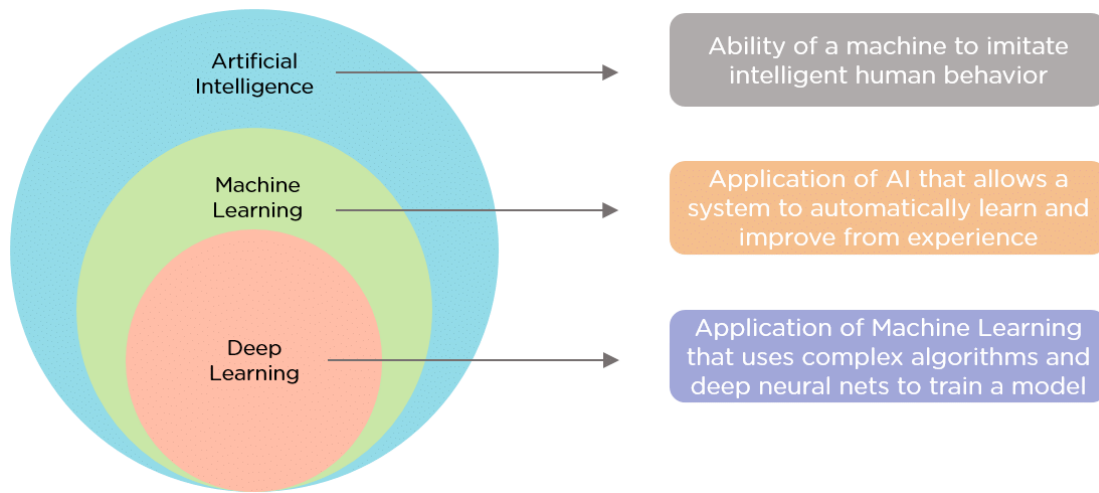
Difference Between Virtual Environment and Conda Environment

Feature	Virtual Environment	Conda Environment
Package Manager	pip	Conda
Python Versions	Uses installed Python	Can install different Python versions
Non-Python Packages	No	Yes
Best For	Python projects	Data Science and AI projects

Conclusion

Virtual environments and Conda environments help developers manage project dependencies efficiently. They keep projects isolated, prevent version conflicts, and improve software development and deployment.

Part D – Introduction to AI & Machine Learning



Question 1: Define Artificial Intelligence

Definition

Artificial Intelligence (AI) is the ability of computers or machines to perform tasks that normally require human intelligence, such as learning, reasoning, problem-solving, and decision-making.

Examples

- Voice assistants (Siri, Google Assistant)
- Chatbots
- Self-driving cars
- Face recognition systems

Question 2: Differentiate AI, Machine Learning, and Deep Learning

Definition

Artificial Intelligence (AI):

Artificial Intelligence is the broad field of creating intelligent systems that can perform tasks such as reasoning, learning, planning, and decision-making similar to humans.

Machine Learning (ML):

Machine Learning is a subset of AI that enables computers to learn patterns from data and make predictions or decisions without being explicitly programmed for every task.

Deep Learning (DL):

Deep Learning is a subset of Machine Learning that uses artificial neural networks with multiple layers to learn complex patterns from large amounts of data. It is especially useful for image recognition, speech recognition, and natural language processing.

Comparison Table

Feature	Artificial Intelligence (AI)	Machine Learning (ML)	Deep Learning (DL)
Meaning	Makes machines perform intelligent tasks.	Enables computers to learn from data.	Uses deep neural networks to learn complex patterns.
Scope	Broadest field.	Subset of AI.	Subset of Machine Learning.
Learning Method	Uses rules, logic, and learning algorithms.	Learns from structured or labeled data.	Learns automatically from large datasets.
Data Requirement	Low to High	Medium to High	Very High
Human Intervention	High	Moderate	Low
Examples	Chatbots, Robots, Virtual Assistants	Spam Detection, Recommendation Systems	Image Recognition, Speech Recognition, Self-driving Cars

Question 3: Explain Supervised, Unsupervised, and Reinforcement Learning

Supervised Learning

Supervised learning uses labeled data to train a model. The model learns from input-output pairs to make predictions.

Example: Email spam detection.

Unsupervised Learning

Unsupervised learning uses unlabeled data to discover hidden patterns or group similar data.

Example: Customer segmentation.

Reinforcement Learning

Reinforcement learning trains an agent through rewards and penalties. The agent learns the best actions by interacting with its environment.

Example: Self-driving cars and game-playing AI.

Question 4: Describe the Machine Learning Pipeline

Definition

A **Machine Learning (ML) pipeline** is a series of steps used to build, train, evaluate, and deploy a machine learning model. It ensures that data is processed correctly and the model performs well on new data.

Steps of the Machine Learning Pipeline

1. Problem Definition

The first step is to clearly identify the problem that needs to be solved and define the project goals. Understanding the problem helps in choosing the right data and machine learning approach.

Example: Predict whether an email is spam or not.

2. Data Collection

Data is collected from different sources such as databases, websites, sensors, APIs, or surveys. The quality and quantity of the collected data play an important role in building an accurate model.

Example: Collecting customer purchase records for sales prediction.

3. Data Cleaning

Raw data often contains missing values, duplicate records, or incorrect information. Data cleaning removes these issues so the dataset becomes accurate and ready for analysis.

Example: Removing duplicate records and filling missing values.

4. Exploratory Data Analysis (EDA)

Exploratory Data Analysis is performed to understand the dataset by finding patterns, relationships, and trends using statistics and visualizations. This helps identify useful information before building the model.

Example: Creating graphs to analyze customer age distribution or sales trends.

5. Feature Engineering

Feature engineering involves selecting, creating, or transforming important variables (features) that help improve the performance of the machine learning model.

Example: Extracting a customer's age from their date of birth or creating a "total purchase amount" feature.

6. Model Selection

In this step, a suitable machine learning algorithm is selected based on the type of problem and available data. Different algorithms may produce different results, so choosing the right one is important.

Example: Selecting Decision Tree, Random Forest, or Linear Regression depending on the task.

7. Training

The selected model is trained using the prepared dataset. During training, the model learns patterns and relationships from the data to make future predictions.

Example: Training a Decision Tree model using historical customer data.

8. Evaluation and Tuning

After training, the model is tested using unseen data to measure its performance. If necessary, model parameters are adjusted (tuned) to improve accuracy and reduce errors.

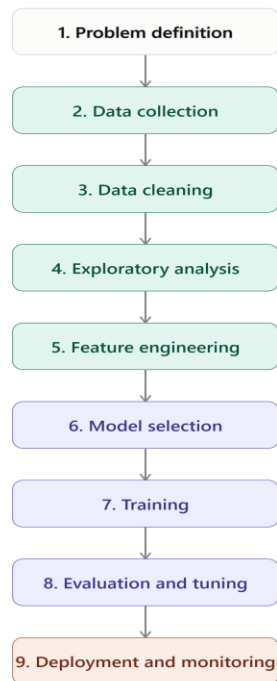
Example: Evaluating the model using accuracy, precision, recall, or F1-score and improving it through parameter tuning.

9. Deployment and Monitoring

Once the model performs well, it is deployed into a real-world application where users can use it. The model is continuously monitored to ensure it performs correctly, and it is updated whenever new data becomes available.

Example: Deploying a product recommendation system on an e-commerce website and monitoring its performance over time.

Machine Learning Pipeline Flow

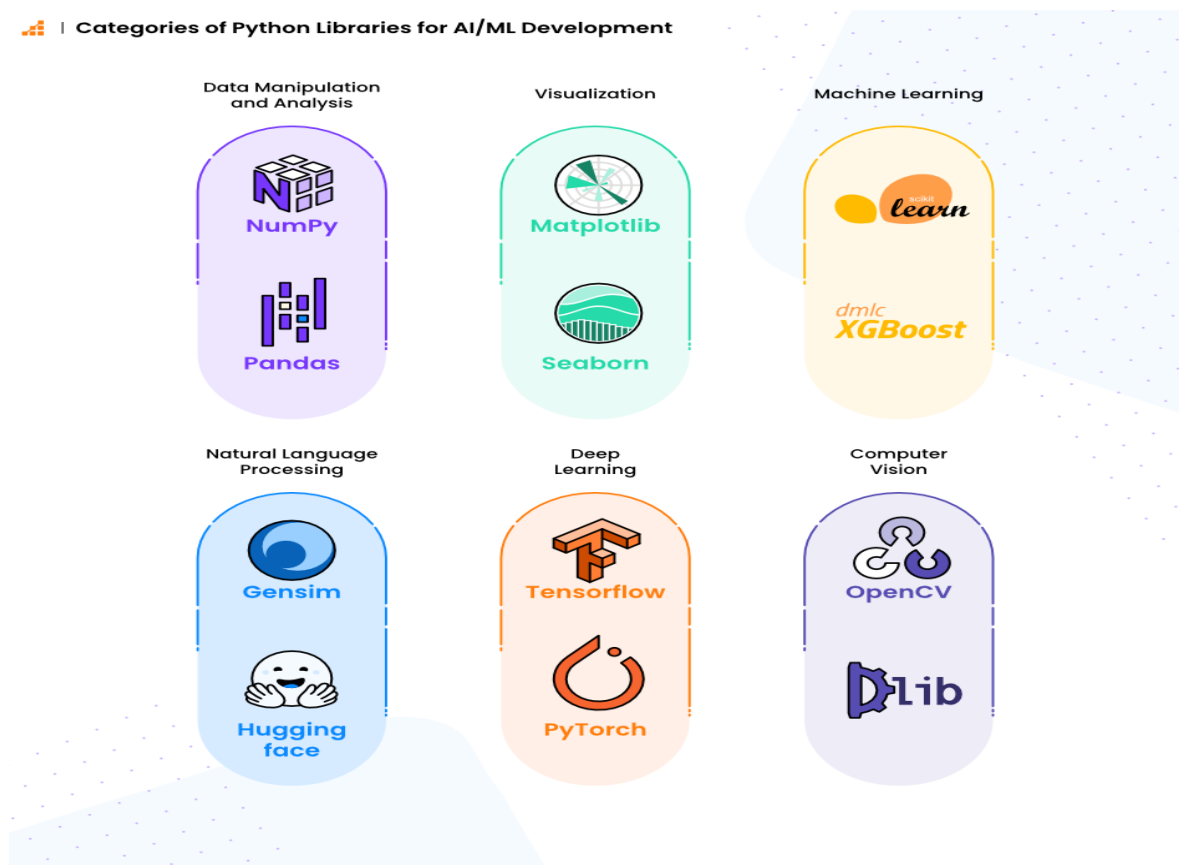


Conclusion

The Machine Learning pipeline provides a structured process for developing reliable and accurate machine learning models. Following these steps helps ensure that models are trained on quality data, evaluated properly, and maintained effectively after deployment.

Part E – AI Libraries Research

 | Categories of Python Libraries for AI/ML Development



Definition

AI libraries are pre-built software libraries that provide tools and functions for developing Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL) applications. These libraries simplify data processing, model building, training, and evaluation.

Comparison of AI Libraries

Library	Purpose	Applications
NumPy	Provides support for numerical computing using multidimensional arrays and mathematical operations.	Scientific computing, matrix operations, data processing, numerical analysis.
Pandas	Used for data manipulation and analysis using DataFrames.	Data cleaning, data analysis, handling CSV/Excel files, preprocessing datasets.
Scikit-learn	Provides machine learning algorithms and model evaluation tools.	Classification, regression, clustering, recommendation systems, predictive analytics.
TensorFlow	An open-source deep learning framework developed by Google for building neural networks.	Image recognition, speech recognition, natural language processing (NLP), deep learning models.
PyTorch	An open-source deep learning framework developed by Meta with dynamic computation graphs.	Deep learning research, computer vision, NLP, robotics, AI model development.

Summary

- **NumPy** is mainly used for numerical calculations and array operations.
- **Pandas** is used to organize, clean, and analyze data.
- **Scikit-learn** provides machine learning algorithms for prediction and classification tasks.
- **TensorFlow** is widely used for building and deploying deep learning models.
- **PyTorch** is popular for AI research and developing advanced deep learning applications.

Conclusion

Python offers a wide range of libraries for AI and Machine Learning development. **NumPy** and **Pandas** are essential for data processing, **Scikit-learn** is used for traditional machine learning, while **TensorFlow** and **PyTorch** are powerful frameworks for deep learning. Choosing the appropriate library depends on the project requirements and the type of AI application being developed.